ESCUELA SUPERIOR POLITECNICA DEL CHIMBORAZO

FACULTAD DE INFORMATICA Y ELECTRONICA

CARRERA DE SOFTWARE

Nombre:

Emilio Pesantez – 7359

Tema:

Diseño arquitectónico de la aplicación

Docente:

Ing. Julio Santillán

Curso:

8vo “1”

Riobamba, 21 de abril de 2026

Diseño arquitectónico de la aplicación

1. Enfoque arquitectónico del proyecto

La solución propuesta implementa una arquitectura cliente-servidor de 3 capas. Este enfoque distribuye las responsabilidades del sistema en componentes especializados, donde el cliente consume servicios del servidor y el servidor, a su vez, organiza internamente la lógica del negocio y el acceso a los datos. La decisión responde a la necesidad de manejar múltiples roles de usuario, procesar evaluaciones de transparencia, proteger información sensible y mantener una evolución ordenada del sistema a lo largo del tiempo.

A diferencia de una solución monolítica con lógica mezclada, la arquitectura de 3 capas permite definir límites técnicos claros. La interfaz de usuario se encarga de capturar y mostrar información; la capa intermedia valida, procesa y aplica reglas; y la capa de datos administra la persistencia y consistencia de la información. Esta separación simplifica el mantenimiento, mejora las pruebas y reduce el impacto de cambios futuros.

2. Estructura general de la arquitectura

La arquitectura se organiza en tres capas principales, conectadas a través de interfaces bien definidas. El usuario interactúa con el frontend; el frontend consume servicios REST del backend; y el backend consulta y actualiza la base de datos según las reglas del sistema.

CAPA DE PRESENTACIÓN (CLIENTE)

Interfaz web en React + Vite. Gestiona navegación, dashboards, formularios, filtros y visualización de resultados.

↓ Comunicación HTTP/HTTPS + API REST ↓

Intercambio de solicitudes, respuestas JSON, autenticación y control de acceso.

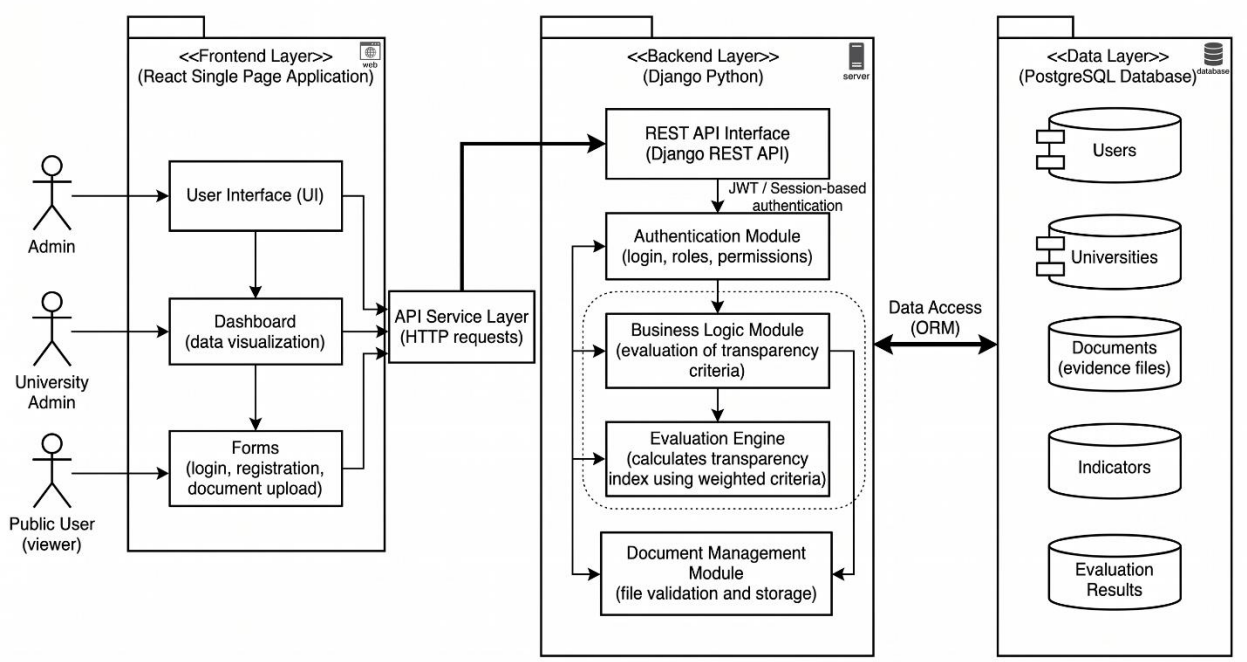
CAPA DE LÓGICA DE NEGOCIO (SERVIDOR)

Backend en Django + DRF. Implementa autenticación, validaciones, evaluación de indicadores, auditoría y reglas del sistema.

CAPA DE DATOS

PostgreSQL. Almacena usuarios, roles, universidades, evidencias, periodos, indicadores, evaluaciones y resultados.

En el siguiente diagrama se muestra la arquitectura general del sistema.



3. Descripción de las capas

3.1 Capa de presentación

Esta capa constituye el punto de interacción entre el usuario y el sistema. Se implementa con React y Vite para ofrecer una experiencia web moderna, dinámica y de alto rendimiento. Su responsabilidad principal es presentar la información de forma clara, capturar acciones del usuario y enviar solicitudes al backend.

- Gestiona el acceso según el rol del usuario, mostrando funcionalidades específicas para administrador general, administrador universitario y auditor o visualizador.
- Presenta dashboards, tablas, formularios, filtros, gráficos y estados de evaluación sin incorporar reglas críticas del negocio en el cliente.
- Consume endpoints REST y procesa las respuestas para actualizar la interfaz de manera reactiva y controlada.

3.2 Capa de lógica de negocio

Esta capa reside en el servidor y concentra las reglas que dan valor al sistema. Se implementa con Django y Django REST Framework, tecnologías adecuadas para desarrollar APIs robustas, seguras y mantenibles. Aquí se controlan procesos clave como autenticación, autorización, validación de evidencias, cálculo de resultados, trazabilidad y manejo de errores.

- Expone servicios REST para registro, inicio de sesión, consulta de indicadores, gestión de evidencias, ejecución de evaluaciones y visualización de resultados.

- Aplica reglas de negocio relacionadas con pesos configurables, cumplimiento de indicadores, validación de estados y generación de reportes o salidas consolidadas.
- Protege operaciones sensibles mediante control de acceso basado en roles y mecanismos de seguridad sobre credenciales y datos personales.

3.3 Capa de datos

La capa de datos se implementa con PostgreSQL y almacena de forma estructurada toda la información requerida por el proyecto. Su diseño relacional favorece la integridad, la consistencia y la trazabilidad de la información, aspectos esenciales en un sistema de evaluación institucional.

- Administra entidades como usuarios, roles, universidades, periodos de evaluación, indicadores, evidencias, resultados finales y retroalimentación.
- Permite mantener relaciones claras entre universidades, criterios evaluados y resultados obtenidos, facilitando consultas seguras y reportes consistentes.
- Asegura persistencia, integridad referencial y soporte para auditoría histórica de acciones y cambios.

4. Flujo operativo entre capas

Paso	Capa involucrada	Acción realizada
1	Usuario / Cliente	El usuario accede al sistema, inicia sesión y selecciona una funcionalidad como cargar evidencias, revisar indicadores o consultar resultados.
2	Presentación	La interfaz captura la acción, valida datos básicos del formulario y envía una solicitud HTTP/HTTPS al backend.
3	Lógica de negocio	El servidor autentica al usuario, verifica permisos, valida la petición y ejecuta las reglas correspondientes.
4	Datos	El backend consulta o actualiza la base de datos según la operación requerida y recupera la información asociada.
5	Lógica de negocio	El servidor procesa el resultado, genera la respuesta estructurada y registra eventos relevantes para auditoría.

5. Justificación explícita de las decisiones arquitectónicas

- La selección de estilos y patrones no fue arbitraria; se realizó con base en el alcance académico del proyecto, la necesidad de seguridad, la existencia de múltiples roles, la

generación de dashboards y la trazabilidad exigida por un sistema de evaluación de transparencia. Por ello, la arquitectura combina decisiones de alto nivel con patrones de implementación que refuerzan modularidad y control.

- Arquitectura en 3 capas: fue seleccionada porque responde directamente a los requisitos del proyecto. La capa de presentación resuelve interacción y visualización, la capa de negocio concentra autenticación, evaluación y auditoría, y la capa de datos garantiza persistencia e integridad. Esta separación reduce acoplamiento y mejora mantenibilidad, escalabilidad y seguridad.
- Cliente-servidor desacoplado: se adoptó este estilo para independizar la evolución del frontend y del backend. Esto permite modificar la interfaz sin alterar la lógica del negocio y viceversa, facilita pruebas por separado y simplifica el despliegue progresivo del sistema.
- SPA + API REST: esta combinación se eligió porque el proyecto requiere dashboards interactivos, navegación fluida, formularios dinámicos y consultas frecuentes de resultados. La SPA mejora la experiencia de uso, mientras que la API REST permite interoperabilidad y deja abierta la posibilidad de incorporar otros clientes o integraciones futuras.
- RBAC y patrones de soporte: el control de acceso basado en roles se justifica porque el sistema distingue claramente entre administrador, administrador universitario y visualizador o auditor. Complementariamente, se adoptan patrones como Provider/Context, Route Guard y Service Layer para centralizar el estado de autenticación, proteger rutas y desacoplar la comunicación con la API.

Patrón o estilo	**Justificación**
Cliente-servidor desacoplado	Permite separar despliegue, mantenimiento y evolución del frontend y backend.
SPA	Mejora la experiencia de uso para dashboards, formularios y navegación sin recargas completas.
API REST	Estandariza la comunicación entre capas y facilita integración futura con otros consumidores.
Provider/Context	Evita prop drilling y centraliza la sesión del usuario en el frontend.
Service Layer	Aísla transporte HTTP y servicios internos, reduciendo acoplamiento entre componentes.
MVT adaptado a JSON	Aprovecha la estructura de Django sin usar plantillas del lado servidor.

6. Relación con los requisitos del proyecto

Necesidad del proyecto	Respuesta de la arquitectura
Gestión de múltiples roles	La capa de negocio centraliza autenticación y autorización, mientras la capa de presentación adapta la interfaz según permisos.
Evaluación de indicadores	Las reglas y cálculos se ejecutan en el servidor, evitando inconsistencias en el cliente.
Almacenamiento de evidencias y resultados	La capa de datos organiza la persistencia con relaciones consistentes y trazables.
Visualización de dashboards	El frontend consume datos procesados por el backend y los presenta de forma clara y dinámica.
Seguridad de credenciales y datos	El servidor aplica mecanismos de protección y evita exponer información crítica en el navegador.

7. Lineamientos de implementación

Para que la arquitectura cliente-servidor de 3 capas funcione correctamente en la práctica, es importante mantener la separación de responsabilidades durante el desarrollo y el despliegue del sistema. Esto implica evitar que el frontend incorpore reglas críticas del negocio, impedir accesos directos no controlados a la base de datos y documentar claramente los contratos de la API.

- Mantener endpoints REST bien definidos, versionados y documentados para facilitar integración y mantenimiento.
- Aplicar control de acceso basado en roles desde el backend, sin depender únicamente de restricciones visuales en el frontend.
- Centralizar validaciones, cálculo de resultados y auditoría en la capa de lógica de negocio.
- Diseñar la base de datos con integridad referencial, índices apropiados y trazabilidad de cambios relevantes.
- Preparar el sistema para producción mediante variables de entorno, configuración segura, HTTPS y políticas de respaldo.

8. Conclusión

En síntesis, el proyecto implementa una arquitectura cliente-servidor de 3 capas porque esta estructura ofrece el equilibrio adecuado entre organización, seguridad, escalabilidad y facilidad de mantenimiento. La capa de presentación atiende la experiencia del usuario; la capa de lógica de negocio concentra las reglas del sistema y los controles de seguridad; y la capa de datos garantiza persistencia e integridad. Gracias a esta distribución, la aplicación puede crecer de forma ordenada y responder con solidez a los objetivos de evaluar la transparencia institucional en universidades públicas.

Por ello, la arquitectura seleccionada no solo satisface los requisitos actuales del sistema, sino que también establece una base técnica robusta para futuras mejoras, como nuevos módulos de análisis, reportes más avanzados, integraciones externas y fortalecimiento de mecanismos de auditoría y seguridad